

Documentação Técnica: Game Wishlist Tracker

1. Visão Geral do Projeto

O **Game Wishlist Tracker** é uma aplicação web full-stack desenvolvida para monitorar preços de jogos desejados pelo usuário (como Hogwarts Legacy ou Phasmophobia, por exemplo). O sistema permite o cadastro de jogos, definição de preços-alvo e visualização em uma interface moderna e responsiva.

2. Stack Tecnológica

Camada	Tecnologia	Justificativa
Front-end / Back-end	Next.js (App Router) + React	Renderização híbrida, rotas de API nativas e facilidade de deploy.
Linguagem	TypeScript	Segurança de tipagem e prevenção de erros em tempo de desenvolvimento.
Estilização	Tailwind CSS	Desenvolvimento ágil de UI com classes utilitárias.
Banco de Dados	Supabase (PostgreSQL)	Banco de dados relacional robusto com API instantânea.
Hospedagem	Vercel	Integração nativa com Next.js e CI/CD automatizado.

3. Estrutura de Diretórios (Padrão Enterprise)

O projeto adota o isolamento do código-fonte utilizando o diretório `src /` para separar as lógicas da aplicação das configurações da raiz, padrão amplamente adotado pelo mercado.

```
/game-wishlist-tracker
├── /src
│   ├── /app           # Rotas e páginas (Next.js App Router)
│   ├── /components    # Componentes React reutilizáveis
│   ├── /lib           # Configurações de bibliotecas (ex: Supabase)
│   └── /types         # Definições de interfaces TypeScript
├── .env.local         # Variáveis de ambiente sensíveis
├── tailwind.config.ts # Configuração de estilo
└── tsconfig.json      # Configuração do TypeScript
```

4. Estrutura de Banco de Dados (Schema)

A tabela principal armazenará as informações dos jogos monitorados.

- **Tabela:** wishlist
- **Colunas:**
 - id (UUID, Primary Key) - Identificador único do registro.
 - game_title (VARCHAR) - Nome do jogo.
 - target_price (DECIMAL) - Preço desejado pelo usuário.
 - current_price (DECIMAL) - Preço atual no mercado.
 - created_at (TIMESTAMPTZ) - Data de inserção no sistema.

5. Convenções de Desenvolvimento e Qualidade

1. **Commits:** Utilizar Conventional Commits (ex: feat: adiciona tabela supabase) para manter o histórico do Git limpo e rastreável.
2. **Segurança:** Chaves de API e strings de conexão de banco de dados devem residir exclusivamente no arquivo `.env.local` e ser declaradas no `.gitignore`.
3. **Componentização:** Componentes de UI genéricos (botões, inputs, modais) devem ser isolados na pasta `src/components` para reuso em toda a aplicação.

6. Análise de Requisitos

Requisitos Funcionais (User Stories)

- **RF01:** Como usuário, quero cadastrar o nome de um jogo e o preço desejado para que eu possa monitorar a minha wishlist.
- **RF02:** Como usuário, quero visualizar uma lista de todos os jogos cadastrados para acompanhar os preços.
- **RF03:** Como usuário, quero poder deletar um jogo da lista caso eu já o tenha comprado ou perdido o interesse.
- **RF04:** Como usuário, quero atualizar o "preço atual" do jogo para saber se ele atingiu a minha meta.

Requisitos Não-Funcionais

- **RNF01:** O sistema deve ser uma Single Page Application (SPA) construída com Next.js para garantir navegação rápida.
- **RNF02:** A interface deve ser totalmente responsiva (Mobile First), utilizando Tailwind CSS.
- **RNF03:** Os dados devem ser persistidos em um banco de dados relacional PostgreSQL hospedado no Supabase.

7. Ciclo de Desenvolvimento (Sprints)

O desenvolvimento foi dividido em 3 Sprints focadas para entregas iterativas.

Sprint 1: Fundação e Banco de Dados

Objetivo: Preparar a base do projeto e a infraestrutura de dados.

- ☒ Inicializar repositório Next.js com Tailwind e TypeScript.
- ☒ Configurar estrutura de diretórios Enterprise (`src/`).
- ☐ Criar projeto no Supabase e gerar chaves de API (URL e ANON_KEY).
- ☐ Criar tabela `wishlist` no banco de dados e inserir dados mockados.
- ☐ Configurar as chaves no arquivo `.env.local`.

Sprint 2: Front-end e Interface (A "Casca")

Objetivo: Criar o visual da aplicação antes de conectar com os dados reais.

- ☐ Criar o componente de Card para exibir o jogo e os preços.
- ☐ Criar o formulário simples para adicionar um novo jogo.
- ☐ Estilizar o layout principal (Header, grid de jogos) usando Tailwind.

Sprint 3: Integração e Lógica (O "Motor")

Objetivo: Fazer a interface conversar com o banco de dados.

- ☐ Instalar a biblioteca `@supabase/supabase-js`.
- ☐ Criar as funções de CRUD (Criar, Ler, Deletar jogos) puxando os dados do Supabase.
- ☐ Conectar o formulário do front-end à função de inserção.
- ☐ Testar fluxo completo e fazer o Deploy final na Vercel.